POC 01 / 04

Blog REST API

POC 1 — PHP 8 Fundamentals, OOP & Version Control with Git

PHP Trainee Programme | Proof of Concept Document | 2026

1. Project Overview

This Proof of Concept challenges you to design and build a fully functional Blog REST API from scratch using core PHP 8 — without any framework. You will apply PHP fundamentals, Object-Oriented Programming principles, and Git/GitHub version control workflows that reflect how professional development teams operate.

By completing this POC, you will demonstrate that you can architect a multi-layer PHP application, write clean and testable OOP code, manage a project through Git branches and pull requests, and deliver a working API that follows REST conventions.

Difficulty	Beginner — Intermediate	**Estimated Time**	3 – 5 days
PHP Version	PHP 8.1+	**Database**	MySQL / SQLite

2. Skills Assessed

Domain	Topics Covered	Course Reference
PHP Fundamentals	Types, arrays, functions, error handling, PDO	Full PHP Tutorial
OOP	Classes, interfaces, traits, inheritance, encapsulation	OOP in PHP
Git & GitHub	Branching, commits, PRs, merge conflicts, .gitignore	Git Crash Course
REST API Design	HTTP verbs, status codes, JSON responses, routing	Professional PHP Dev
Database	PDO prepared statements, migrations, CRUD operations	Full PHP Tutorial
Code Quality	PSR-12 coding standard, DRY, SOLID principles	Professional PHP Dev

3. Functional Requirements

Your API must expose the following endpoints. All responses must be JSON. All errors must return appropriate HTTP status codes.

Method	Endpoint	Description
GET	/api/posts	Return paginated list of all blog posts
GET	/api/posts/{id}	Return a single post by ID — 404 if not found
POST	/api/posts	Create a new post — validate all required fields
PUT	/api/posts/{id}	Full update of an existing post
PATCH	/api/posts/{id}	Partial update (e.g., change status only)

DELETE	/api/posts/{id}	Soft-delete a post (set deleted_at, do not remove row)
GET	/api/posts/{id}/comments	Return all comments for a post
POST	/api/posts/{id}/comments	Add a comment to a post

4. Technical Architecture

Your project must follow this directory structure. No framework is allowed — you must build the router, request handler, and response classes yourself.

Project Structure

```
blog-api/
├── public/
│   └── index.php           ← Entry point — all requests routed here
├── src/
│   ├── Controllers/
│   │   ├── PostController.php
│   │   └── CommentController.php
│   ├── Models/
│   │   ├── Post.php
│   │   └── Comment.php
│   ├── Core/
│   │   ├── Router.php      ← Custom HTTP router
│   │   ├── Request.php     ← Wraps $_SERVER, php://input
│   │   ├── Response.php    ← JSON response builder
│   │   └── Database.php    ← PDO singleton
│   └── Exceptions/
│       ├── NotFoundException.php
│       └── ValidationException.php
├── database/
│   └── migrations/         ← SQL migration files
├── tests/                  ← Unit tests (PHPUnit)
├── .gitignore
├── composer.json
└── README.md
```

4.1 Required OOP Patterns

Pattern	Where to Apply It
Singleton	Database connection class — only one PDO instance allowed
Repository Pattern	PostRepository.php — separates data access from business logic
Interface	Declare a RepositoryInterface — PostRepository implements it
Abstract Class	BaseModel — shared properties and methods for all models
Trait	Timestampable trait — adds created_at / updated_at to any model
Exception Handling	Custom exceptions caught in index.php, returned as JSON errors

5. Git & GitHub Workflow Requirements

You must demonstrate professional version control practice throughout this POC. Your commit history, branch structure, and pull requests are part of the assessment.

> REQUIREMENT: All code must live in a GitHub repository. You must share the repository link in your submission. A single commit with all code will not be accepted.

Required Branch Strategy

Branch	Purpose
main	Production-ready code only — never commit directly to main
develop	Integration branch — merge feature branches here first
feature/router	Build the custom HTTP router
feature/post-crud	Implement Post model, controller, repository
feature/comments	Implement Comment endpoints
feature/validation	Add input validation and custom exceptions

Commit Message Convention

Conventional Commits

```
# Format: <type>(<scope>): <short description>

feat(router):    add support for PATCH method routing
fix(post):       return 404 when post is soft-deleted
refactor(db):    extract query builder into BaseRepository
test(post):      add unit tests for PostRepository::findById
docs(readme):    document API endpoints with curl examples
chore(composer): add PHPUnit dependency
```

Pull Request Requirements

- Every feature branch must be merged via a Pull Request — not a direct push
- Each PR must have a description explaining what was built and why
- At least 2 PRs must include a self-review comment on your own code
- Resolve at least one intentional merge conflict between two branches

6. Code Quality Standards

All submitted code must comply with the following standards. Reviewers will check these during evaluation.

PHP 8.1 — PSR-12 Compliant Example

```
<?php
```

```php
declare(strict_types=1);  // Required at top of every file

namespace App\Models;

use App\Core\Database;
use App\Exceptions\NotFoundException;

/**
 * Post model — represents a blog post entity.
 *
 * @property int    $id
 * @property string $title
 * @property string $body
 * @property string $status   draft|published|archived
 */
final class Post extends BaseModel
{
    protected string $table = 'posts';

    public function __construct(
        private readonly string $title,
        private readonly string $body,
        private string $status = 'draft',
    ) {}

    public function publish(): void
    {
        $this->status = 'published';
    }
}
```

7. Acceptance Criteria

Criterion	Pass Condition
All 8 API endpoints functional	Tested via Postman — all return correct HTTP status codes
OOP patterns applied	Singleton, Repository, Interface, Abstract, Trait all present
PSR-12 compliance	No PSR-12 violations — run: ./vendor/bin/phpcs --standard=PSR12 src/
Input validation	Missing/invalid fields return HTTP 422 with error details in JSON
Soft delete working	Deleted posts excluded from GET /api/posts but row exists in DB
Git history	Minimum 15 meaningful commits across at least 4 feature branches
Pull Requests	Minimum 3 PRs with descriptions — merged into develop, then main
README complete	Setup instructions, endpoint docs, and example curl commands present
PHPUnit tests	Minimum 5 unit tests covering model and repository methods

8. Submission Requirements

1. GitHub repository link — must be public or shared with the reviewer
2. Postman Collection (.json) exported with all 8 endpoints pre-configured
3. Screenshot of GitHub pull request list showing merged branches
4. Screenshot of PHPUnit test output (all tests passing)
5. Short reflection (max 200 words): What OOP pattern did you find most useful and why?

> Your submission will not be reviewed without the GitHub link. Make sure the repository is accessible before your submission deadline.